Name: Janelit Cabus. **Section: D.** **Date:** 11/10/26

Exercise Title: Campus Café Ordering System

1.Explanation of OOP Concepts Used

Conditional Statements: I used if, if-else, and if-else-if statements to check conditions and make decisions. These validate prices, check bever

Classes and Objects: I created seven Java classes: MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp. Each class has its own responsibility, and objects are created in the main method to demonstrate the system.

Attributes and Behaviors: Attributes store information such as customer names, item prices, sizes, and loyalty points. Behaviors are methods that perform actions such as calculating prices, adding items, and printing receipts.

Object Interaction: The Order class connects a Customer object with MenuItem objects. It manages the customer's items, calculates the amount due, prints the receipt, and updates loyalty points.

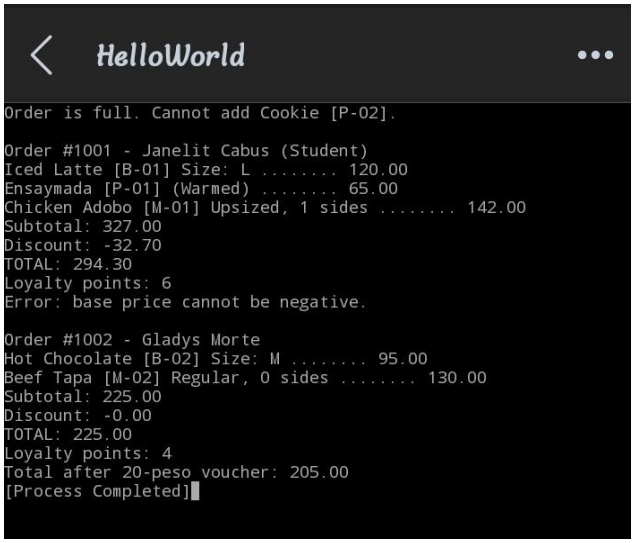
Constructors: I used default and parameterized constructors to initialize objects. Constructor overloading is demonstrated in the Customer class through three constructors with different parameters.

Encapsulation: I applied encapsulation by declaring instance variables as private or protected. Getters and setters control access to the data, while validation prevents invalid values.

Polymorphism: I applied polymorphism through method overloading and method overriding. Overloading appears in the Customer and Order classes, while Beverage, Pastry, and Meal override methods to calculate their prices differently.

Abstraction: I created an abstract class named MenuItem with an abstract computePrice() method. Each subclass provides its own implementation of the pricing method.

Java Naming Conventions: I used PascalCase for class names and camelCase for variables and methods. I also used meaningful names to make the program easier to read and understand.



3. Program Output

The program successfully demonstrates how the Campus Café Ordering System processes customer orders, calculates prices, applies discounts, and awards loyalty points.

First Order – Order #1001

The first customer is Janelit Cabus, who is registered as a student. The customer ordered an Iced Latte (Large) for ₱120, an Ensaymada (Warmed) for ₱65, and Chicken Adobo (Upsized with 1 side) for ₱142. The subtotal is ₱327.00. A student discount of ₱32.70 was applied, making the final total ₱294.30. The customer also earned 6 loyalty points.

The program also displays two error messages. First, it prevents adding a Cookie because the order is already full. Second, it prevents a menu item from having a negative base price. This shows that the program has validation to avoid invalid orders and prices.

Second Order – Order #1002

The second customer is Gladys Morte. She ordered a Hot Chocolate (Medium) for ₱95 and Beef Tapa (Regular, with no additional sides) for ₱130. The subtotal is ₱225.00. No discount was applied, so the final total remains ₱225.00. The customer earned 4 loyalty points. After applying a ₱20 voucher, the final amount became ₱205.00.

4. Challenges Encountered and Solutions

While developing the program, I found it challenging to understand how the different classes interact with one another. I also needed to make sure that inheritance, method overriding, and constructors were used correctly. Another challenge was validating data and limiting each order to three items. I addressed these problems by reviewing the instructions, separating the code into individual Java files, using if-else statements for validation, and testing the program to check whether prices, discounts, and loyalty points were calculated correctly.

5. Conclusion

Through this laboratory exercise, I improved my understanding of Java Object-Oriented Programming. I learned how to create classes and objects, use constructors, apply inheritance and polymorphism, protect data through encapsulation, and use abstraction. Overall, this activity helped me improve my programming skills and understand how OOP concepts can be applied in a simple ordering system.